

Page Replacement Algorithms

1. Why Page Replacement Is Needed

Modern operating systems use **virtual memory** so that a process can run even when its complete program is not loaded into RAM.

Instead of loading the entire process into physical memory, the OS divides memory into fixed-size blocks:

- **Page**: fixed-size block of a process in virtual memory.
- **Frame**: fixed-size block of physical memory.
- **Page table**: data structure that maps virtual pages to physical frames.

When a process refers to a page that is not currently present in main memory, a **page fault** occurs.

If there is a free frame available, the OS simply loads the required page into that frame.

But if all frames are already occupied, the OS must choose one existing page to remove from memory. This decision is made by a **page replacement algorithm**.

In simple words:

Page replacement decides which page should be removed from RAM when a new page has to be loaded and memory is full.

2. Important Terms

Page Fault

A **page fault** occurs when the CPU refers to a page that is not currently present in main memory.

Example:

If the process needs page 5, but page 5 is not loaded into any frame, then a page fault occurs.

The OS then:

1. Traps to the operating system.
2. Checks whether the memory reference is valid.
3. Finds the required page on disk.
4. Selects a free frame or chooses a victim page.
5. Loads the required page into memory.
6. Updates the page table.
7. Restarts the interrupted instruction.

Page Hit

A **page hit** occurs when the required page is already present in main memory.

Reference String

A **reference string** is the sequence of page numbers accessed by a process.

Example:

```
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2
```

This means the process accesses page 7, then page 0, then page 1, and so on.

Frame

A **frame** is one slot in physical memory where one page can be stored.

If a question says "number of frames = 3", it means only 3 pages can be kept in RAM at a time.

Victim Page

The **victim page** is the page selected for replacement when memory is full.

3. Goals of a Page Replacement Algorithm

A good page replacement algorithm should:

- Minimize the number of page faults.
- Avoid removing pages that will be used soon.
- Be simple enough to implement efficiently.
- Use available hardware support when possible.
- Avoid excessive overhead.
- Prevent thrashing.

The main goal is:

Reduce page faults and improve system performance.

4. FIFO Page Replacement

FIFO stands for **First-In, First-Out**.

It is the simplest page replacement algorithm.

Main Idea

The page that entered memory first is removed first.

FIFO treats memory like a queue:

- New pages are inserted at the rear.
- Oldest pages are removed from the front.

It does not care how frequently or recently a page is used.

FIFO Algorithm

For each page in the reference string:

1. If the page is already in memory, it is a page hit.
2. If the page is not in memory:
 - Page fault occurs.
 - If a free frame is available, insert the page.
 - If memory is full, remove the oldest loaded page.
 - Insert the new page.

Example

Reference string:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Number of frames:

3

FIFO simulation:

Reference	Frames After Access	Hit/Fault	Explanation
1	1 - -	Fault	1 loaded
2	1 2 -	Fault	2 loaded
3	1 2 3	Fault	3 loaded
4	2 3 4	Fault	1 was oldest, replaced
1	3 4 1	Fault	2 was oldest, replaced
2	4 1 2	Fault	3 was oldest, replaced
5	1 2 5	Fault	4 was oldest, replaced
1	1 2 5	Hit	1 already present
2	1 2 5	Hit	2 already present
3	2 5 3	Fault	1 was oldest, replaced
4	5 3 4	Fault	2 was oldest, replaced
5	5 3 4	Hit	5 already present

Total page faults:

Advantages of FIFO

- Very simple to understand.
- Easy to implement using a queue.
- Low overhead.
- Does not require hardware support.

Disadvantages of FIFO

- May remove heavily used pages.
- Does not consider recent or future usage.
- Can give poor performance.
- Suffers from Belady's anomaly.

Belady's Anomaly

Usually, increasing the number of frames should reduce the number of page faults.

But in FIFO, sometimes increasing the number of frames can increase the number of page faults.

This strange behavior is called **Belady's anomaly**.

Example:

For some reference strings, FIFO may produce more page faults with 4 frames than with 3 frames.

Important placement point:

FIFO can suffer from Belady's anomaly. LRU and Optimal do not suffer from Belady's anomaly because they are stack algorithms.

5. LRU Page Replacement

LRU stands for **Least Recently Used**.

Main Idea

LRU replaces the page that has not been used for the longest time in the past.

It is based on the principle of locality:

If a page has been used recently, it is likely to be used again soon.

LRU Algorithm

For each page reference:

1. If the page is already in memory:
 - It is a hit.

- Update its recent usage information.
- 2. If the page is not in memory:
 - Page fault occurs.
 - If a free frame exists, load the page.
 - If memory is full, replace the least recently used page.

Example

Reference string:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2

Number of frames:

3

LRU simulation:

Reference	Frames After Access	Hit/Fault	Explanation
7	7 - -	Fault	7 loaded
0	7 0 -	Fault	0 loaded
1	7 0 1	Fault	1 loaded
2	2 0 1	Fault	7 least recently used
0	2 0 1	Hit	0 recently used
3	2 0 3	Fault	1 least recently used
0	2 0 3	Hit	0 recently used
4	4 0 3	Fault	2 least recently used
2	4 0 2	Fault	3 least recently used
3	4 3 2	Fault	0 least recently used
0	0 3 2	Fault	4 least recently used
3	0 3 2	Hit	3 already present
2	0 3 2	Hit	2 already present

Total page faults:

9

How LRU Can Be Implemented

LRU is conceptually simple, but exact implementation can be expensive.

Common implementation methods:

1. Counter Method

The OS maintains a counter or timestamp.

Whenever a page is accessed:

- Current time is stored with that page.
- When replacement is needed, page with the smallest timestamp is removed.

Problem:

- Searching for the smallest timestamp may take time.
- Requires updating metadata on every memory reference.

2. Stack Method

Maintain pages in a stack-like structure:

- Recently used pages move to the top.
- Least recently used page remains at the bottom.

Problem:

- Updating the stack on every memory reference is expensive.

3. Hardware Support

Some systems use reference bits, counters, or special hardware support to approximate LRU.

Exact LRU is often costly, so real operating systems usually use approximations such as Clock or Second Chance.

Advantages of LRU

- Better performance than FIFO in many cases.
- Uses past behavior to predict future behavior.
- Does not suffer from Belady's anomaly.
- Works well with locality of reference.

Disadvantages of LRU

- More complex than FIFO.
- Exact LRU implementation is expensive.
- Requires extra hardware/software support.
- Updating usage information on every reference can be costly.

6. Optimal Page Replacement

Optimal page replacement is also called:

- OPT
- MIN
- Belady's optimal algorithm

Main Idea

Optimal replaces the page that will not be used for the longest time in the future.

In simple words:

Remove the page whose next use is farthest away.

If a page will never be used again, it is the best page to replace.

Optimal Algorithm

For each page reference:

1. If the page is already in memory, it is a hit.
2. If the page is not in memory:
 - Page fault occurs.
 - If a free frame exists, load the page.
 - If memory is full:
 - Look ahead in the future reference string.
 - Find when each page in memory will be used next.
 - Replace the page used farthest in the future.
 - If a page is never used again, replace it.

Example

Reference string:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2

Number of frames:

3

Optimal simulation:

Reference	Frames After Access	Hit/Fault	Explanation
7	7 - -	Fault	7 loaded
0	7 0 -	Fault	0 loaded

Reference	Frames After Access	Hit/Fault	Explanation
1	7 0 1	Fault	1 loaded
2	2 0 1	Fault	7 is never used again
0	2 0 1	Hit	0 already present
3	2 0 3	Fault	1 is never used again
0	2 0 3	Hit	0 already present
4	2 4 3	Fault	0 used later than 2 and 3
2	2 4 3	Hit	2 already present
3	2 4 3	Hit	3 already present
0	2 0 3	Fault	4 is never used again
3	2 0 3	Hit	3 already present
2	2 0 3	Hit	2 already present

Total page faults:

7

Why Optimal Is Important

Optimal gives the minimum possible number of page faults for a given reference string and number of frames.

It is mostly used as a benchmark to compare other page replacement algorithms.

Can Optimal Be Used Practically?

Usually, no.

Optimal requires knowledge of future page references. In real systems, the OS does not know the exact future behavior of a process.

Therefore:

Optimal is theoretically best but practically difficult or impossible to implement exactly.

Advantages of Optimal

- Gives the minimum possible page faults.
- Does not suffer from Belady's anomaly.
- Useful for theoretical comparison.
- Helps judge how close other algorithms are to the best possible performance.

Disadvantages of Optimal

- Requires future knowledge.
- Not practical for real operating systems.
- Mostly used for analysis, not implementation.

7. Clock Page Replacement Algorithm

The Clock algorithm is an efficient approximation of LRU.

It is also known as:

- Second Chance algorithm
- Clock replacement

Why Clock Is Needed

LRU performs well, but exact LRU is expensive because it requires tracking every memory access precisely.

Clock gives pages a "second chance" using a reference bit.

It is simpler and cheaper than exact LRU.

Main Idea

Each page has a **reference bit**:

- Reference bit **1**: page has been used recently.
- Reference bit **0**: page has not been used recently.

Pages are arranged in a circular list, like positions on a clock.

A pointer, called the **clock hand**, moves around the circular list to find a page to replace.

Clock Algorithm

When a page needs to be replaced:

1. Check the page pointed to by the clock hand.
2. If its reference bit is **0**:
 - Replace this page.
 - Move the clock hand to the next frame.
3. If its reference bit is **1**:
 - Set the reference bit to **0**.
 - Give the page a second chance.
 - Move the clock hand to the next frame.
4. Repeat until a page with reference bit **0** is found.

When a page is accessed, its reference bit is set to **1**.

Example

Suppose memory has 4 frames:

```

Frame:           A  B  C  D
Reference bit: 1  0  1  1
Clock hand:     ^
  
```

Replacement needed.

Step-by-step:

1. Clock hand points to **A**.
 - A has reference bit **1**.
 - Set A's bit to **0**.
 - Move ahead.
2. Clock hand points to **B**.
 - B has reference bit **0**.
 - Replace B.

After replacement:

```

Frame:           A  X  C  D
Reference bit: 0  1  1  1
  
```

X is the newly loaded page.

Advantages of Clock

- Efficient approximation of LRU.
- Lower overhead than exact LRU.
- Easy to implement using reference bits.
- Used in many real operating systems or as a basis for real policies.

Disadvantages of Clock

- Not as accurate as true LRU.
- Performance depends on reference-bit behavior.
- If all pages have reference bit **1**, the clock hand may scan many pages.

FIFO vs Clock

Clock is similar to FIFO because it moves in circular order, but it improves FIFO by giving recently used pages a second chance.

FIFO removes the oldest page directly.

Clock checks whether the oldest candidate was recently used before removing it.

8. Thrashing

Definition

Thrashing is a condition in which the system spends more time swapping pages in and out of memory than executing actual instructions.

In simple words:

Thrashing happens when excessive page faults make the system extremely slow.

Why Thrashing Happens

Thrashing usually occurs when:

- Too many processes are loaded into memory.
- Each process gets too few frames.
- The working set of a process does not fit in available memory.
- The degree of multiprogramming is too high.
- Page fault rate becomes very high.

What Happens During Thrashing

When a process does not have enough frames:

1. It accesses a page that is not in memory.
2. A page fault occurs.
3. The OS replaces some page.
4. Very soon, the replaced page is needed again.
5. Another page fault occurs.
6. This repeats continuously.

As a result:

- CPU utilization decreases.
- Disk I/O increases heavily.
- System response time becomes very poor.
- Processes make little actual progress.

Thrashing and CPU Utilization

At first, increasing the degree of multiprogramming improves CPU utilization.

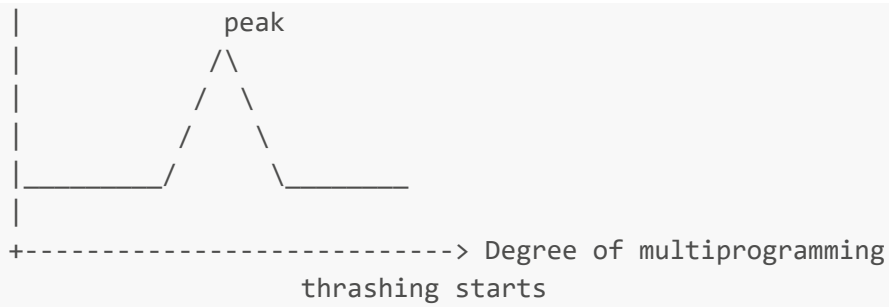
But after a certain point, too many processes compete for memory.

Each process gets fewer frames, causing more page faults.

Eventually, CPU utilization drops sharply because the CPU waits for disk I/O most of the time.

Conceptually:

CPU utilization
^



Symptoms of Thrashing

- Very high page fault rate.
- Low CPU utilization.
- High disk activity.
- Slow system response.
- Processes appear to be stuck.
- OS spends too much time handling page faults.

How to Control Thrashing

Common methods:

- Reduce the degree of multiprogramming.
- Allocate more frames to each process.
- Use working set model.
- Use page fault frequency control.
- Suspend or swap out some processes.
- Improve locality by better program design.

Placement Interview Explanation

If asked "What is thrashing?", a strong answer is:

Thrashing is a state where a process or system spends most of its time servicing page faults and swapping pages instead of executing instructions. It occurs when processes do not have enough frames to hold their active working set, causing very frequent page replacement and poor CPU utilization.

9. Working Set

Definition

The **working set** of a process is the set of pages that the process is actively using during a recent time window.

It is based on the principle of locality.

If a process recently used a group of pages, it is likely to continue using those pages for some time.

Working Set Window

The working set is usually defined using a window size, often represented by delta.

Let:

Delta = fixed number of recent page references

The working set contains all distinct pages referenced in the last **Delta** references.

Example

Reference string:

1, 2, 5, 6, 2, 1, 7, 2, 6, 7, 8

Suppose:

Delta = 5

At the last reference **8**, the last 5 references are:

7, 2, 6, 7, 8

Distinct pages:

{2, 6, 7, 8}

So the working set is:

{2, 6, 7, 8}

Working set size:

4

Working Set Model

The working set model says:

A process should be allocated enough frames to hold its current working set.

If the OS can keep the working set of each active process in memory, page faults remain low.

If not, thrashing may occur.

Total Demand for Frames

For multiple processes:

$$D = WSS1 + WSS2 + WSS3 + \dots + WSSn$$

Where:

- D = total demand for frames
- WSS_i = working set size of process i
- m = total number of available frames

If:

$$D \leq m$$

Then all working sets can fit in memory.

If:

$$D > m$$

Then there are not enough frames, and thrashing may occur.

How Working Set Helps Prevent Thrashing

The OS monitors each process's working set.

If total demand exceeds available memory, the OS can:

- Suspend some processes.
- Reduce the degree of multiprogramming.
- Give more frames to active processes.
- Avoid running too many memory-hungry processes at once.

Choosing the Working Set Window

Choosing the correct value of delta is important.

If delta is too small:

- It may not include the full locality.
- Important pages may be missed.
- Page faults may increase.

If delta is too large:

- It may include pages from old localities.
- Working set size may become unnecessarily large.
- Memory may be wasted.

Locality of Reference

The working set model depends on **locality of reference**.

Locality means a program tends to access a small part of its address space repeatedly for a period of time.

There are two major types:

Temporal Locality

If a page is accessed now, it is likely to be accessed again soon.

Example:

- Loop variables
- Frequently used instructions
- Stack data

Spatial Locality

If a page is accessed now, nearby pages or addresses are likely to be accessed soon.

Example:

- Arrays
- Sequential instruction execution
- Contiguous memory access

10. Page Fault Frequency

Page Fault Frequency, or **PFF**, is another method used to control thrashing.

Main Idea

The OS monitors the page fault rate of each process.

- If page fault rate is too high, the process needs more frames.
- If page fault rate is too low, the process may have more frames than necessary.

How PFF Works

The OS defines:

- Upper bound for page fault rate.
- Lower bound for page fault rate.

If page fault rate goes above the upper bound:

- Allocate more frames to the process.
- Or suspend another process if no frames are available.

If page fault rate goes below the lower bound:

- Remove some frames from the process.

This keeps page fault rate within an acceptable range.

Working Set vs Page Fault Frequency

Feature	Working Set	Page Fault Frequency
Main focus	Pages used recently	Rate of page faults
Based on	Locality	Fault monitoring
Goal	Keep active pages in memory	Keep fault rate acceptable
Prevents thrashing	Yes	Yes
Complexity	Needs tracking recent references	Needs tracking fault rate

11. Comparison of FIFO, LRU, Optimal, and Clock

Algorithm	Replacement Decision	Uses Past?	Uses Future?	Practical?	Belady's Anomaly?
FIFO	Oldest loaded page	No	No	Yes	Yes
LRU	Least recently used page	Yes	No	Approximate versions are practical	No
Optimal	Page used farthest in future	No	Yes	No	No
Clock	First page with reference bit 0	Approximate	No	Yes	No in typical stack-theory sense is not guaranteed like LRU; it is an approximation

12. Which Algorithm Gives Best Performance?

In theory:

Optimal is best.

In practice:

LRU-like approximations such as Clock are commonly used.

Typical performance order for many reference strings:

$\text{Optimal} \leq \text{LRU} \leq \text{Clock/FIFO}$

Here, lower means fewer page faults.

But actual performance depends on the reference string.

13. Common Placement Questions

Q1. What is page replacement?

Page replacement is the process of selecting a page to remove from main memory when a page fault occurs and no free frame is available.

Q2. Why is page replacement needed?

It is needed because physical memory is limited. When memory is full and a required page is not present, the OS must remove an existing page to load the required page.

Q3. Which page replacement algorithm is best?

Optimal is theoretically best because it gives the minimum number of page faults. However, it requires future knowledge, so it is not practical. In real systems, LRU approximations such as Clock are commonly used.

Q4. Why is FIFO not always good?

FIFO removes the oldest page without checking whether it is still heavily used. It may remove important pages and can suffer from Belady's anomaly.

Q5. What is Belady's anomaly?

Belady's anomaly is the situation where increasing the number of page frames increases the number of page faults. FIFO can suffer from this anomaly.

Q6. Why is LRU better than FIFO?

LRU considers recent usage. It replaces the page that has not been used for the longest time, which usually works well because programs show locality of reference.

Q7. Why is Optimal not used in real operating systems?

Optimal requires knowledge of future page references. Since the OS cannot know the exact future, Optimal is not practical.

Q8. What is the Clock algorithm?

Clock is a page replacement algorithm that approximates LRU using a reference bit and a circular list of pages. Pages with reference bit 1 get a second chance; pages with reference bit 0 can be replaced.

Q9. What is thrashing?

Thrashing is a condition where the system spends most of its time handling page faults and swapping pages instead of executing useful instructions.

Q10. How can thrashing be prevented?

Thrashing can be prevented by reducing the degree of multiprogramming, allocating enough frames to processes, using the working set model, or controlling page fault frequency.

Q11. What is a working set?

The working set is the set of pages actively used by a process during a recent time window.

Q12. How does the working set model prevent thrashing?

It ensures that each active process gets enough frames to hold its current working set. If total working set demand exceeds available memory, the OS reduces the degree of multiprogramming.

14. Quick Revision Points

- Page replacement is needed when a page fault occurs and memory has no free frame.
- FIFO removes the oldest loaded page.
- FIFO is simple but can suffer from Belady's anomaly.
- LRU removes the least recently used page.
- LRU uses past behavior and works well due to locality.
- Optimal removes the page whose next use is farthest in the future.
- Optimal gives the minimum page faults but is not practical.
- Clock approximates LRU using a reference bit and circular pointer.
- Thrashing happens when page faults become excessive.
- Working set is the set of pages actively used in a recent time window.
- If total working set demand is greater than available frames, thrashing may occur.
- Page fault frequency control adjusts frame allocation based on fault rate.

15. Short Comparison for Interviews

FIFO:

Simple, queue-based, replaces oldest page, may suffer from Belady's anomaly.

LRU:

Replaces least recently used page, better than FIFO, no Belady's anomaly, costly to implement exactly.

Optimal:

Replaces page used farthest in future, gives minimum faults, theoretical only.

Clock:

Efficient LRU approximation using reference bit and circular pointer, practical and widely used.

Thrashing:

Too many page faults; system spends more time swapping than executing.

Working Set:

Set of pages currently needed by a process; used to allocate enough frames and avoid thrashing.